



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Comparative Analysis of Widely Used Zipfian Distribution Implementations

Rodrigo Felix Forno





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

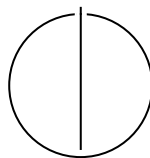
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Comparative Analysis of Widely Used Zipfian Distribution Implementations

**Vergleichende Analyse von
Implementierungen weit verbreiteten
Zipfianverteilung**

Author: Rodrigo Felix Forno
Examiner: Prof. Dr. Viktor Leis
Supervisor: MSc. Bohyun Lee, MSc. Gabriel Haas
Submission Date: 25th of March, 2025



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 25th of March, 2025

Rodrigo Felix Forno

Acknowledgments

First and foremost, I would like to thank my supervisors Gabriel and Lia for their guidance and support throughout the course of this thesis. Their feedback and advice were invaluable in shaping the direction of this work and I am grateful for their patience and understanding. I would also like to thank my family and friends for their encouragement and support during this time, be it with ideas, hearing me out when I needed to talk or simply being there for me. Without them it would have been a lot harder to complete this work. Finally, I would like to thank the TUM Department of Informatics for providing me with the opportunity to pursue this research and take the time to improve my ability as a researcher and learn to contribute to the field of computer science.

Abstract

This thesis analysis common implementations of sampling algorithms used to generate variates of the Zipfian distribution, with special attention given to the use cases found in storage systems and database technology research. Since the late eighties, there has been increased interest in evaluating the performance of those systems under very skewed workloads, which are often modeled using the Zipfian distribution. While the theory behind some sampling techniques for the Zipfian distribution is well grounded, practical implementations vary in their accuracy and performance, which can lead to incorrect results in the evaluation of storage systems. This work presents a comprehensive analysis of the most common sampling algorithms found in benchmarking suites across academia and industry, and offers a comparison in terms of precision, speed and applicability to the aforementioned research areas. Parallely, two programs were written to help analysis: a benchmark tool was developed to run experiments on different implementations and implementation of different variate generators was done in the programming language C++. Evaluation shows that most found implementations are suitable for the use cases in storage systems and database technology research, but some, such as FIO, are to be avoided. It was also revealed that table lookups might be an interesting alternative to established method, but further investigation is required.

Contents

Acknowledgments	iv
Abstract	v
1 Introduction	1
2 Background	3
2.1 The Zipfian Distribution	3
2.2 Recognizing a Zipfian distribution	4
2.3 Discrete Variate Generation	5
2.3.1 Rejection Sampling	6
2.3.2 Rejection-Inversion Sampling	7
2.3.3 Marsaglia’s and Tsang’s Condensed Table Lookup	8
2.3.4 Jim Gray’s Approximative Method	9
2.4 Error Metrics	10
3 Approach	14
3.1 Research Approach	14
3.2 Common Frameworks	15
3.2.1 Yahoo! Cloud Serving Benchmark	16
3.2.2 Flexible I/O Tester	17
3.2.3 Sysbench	18
3.2.4 OLTP-Bench	18
3.3 Benchmarking Module	18
3.3.1 Commands	20
4 Evaluation	21
4.1 Experimental Setup	21
4.2 Methodology	21
4.3 Evaluating accuracy	23
4.4 Evaluating Performance	31
4.4.1 Microbenchmarking	31
4.5 Overall Evaluation	33

Contents

Abbreviations	34
List of Figures	35
List of Tables	36
Bibliography	37

1 Introduction

The Zipfian distribution is a power law that has been observed in a wide range of natural and artificial phenomena. It is characterized by the fact that the frequency of an element is inversely proportional to its rank, meaning the position of the element in the sorted list of elements. A quick example makes this principle quite clear.

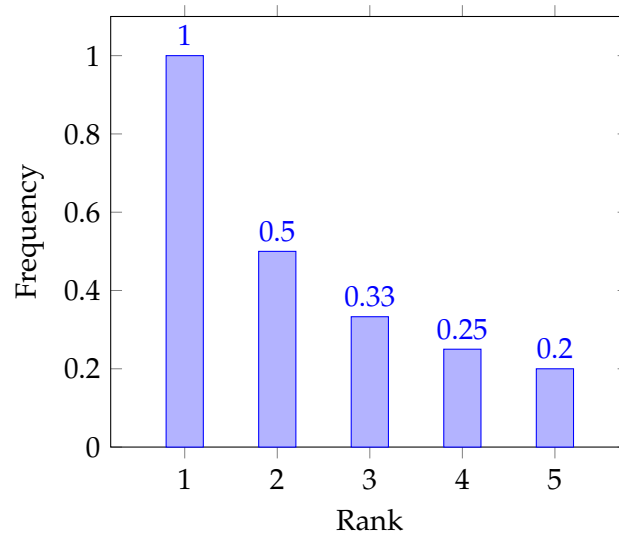


Figure 1.1: Example of a Zipfian distribution

This means that the most frequent element is twice as frequent as the second most frequent element, three times as frequent as the third most frequent element, and so on. This distribution has been observed in a wide range of natural phenomena, such as the distribution of words in natural languages, the distribution of cities by population, among others [1]. And also within the fields of storage systems research, networking theory as well as databases, there has been more compelling evidence that the Zipfian distribution (or more generally, heavy-tailed distributions) is a good model for the distribution of access patterns in these systems [2, 3, 4].

The focus however of this thesis is on the large-scale generation of Zipfian samples, which are essential for the evaluation of latency and general performance in storage systems and databases. To this end, the state-of-the-art in synthetic workload generation

is reviewed and common solutions are identified and evaluated. Parallely, a study of variate generation algorithms is conducted, with a focus on the generation of Zipfian variates. This will allow for the comparison in terms of speed and accuracy of common implementations as well as give a baseline for an improved version. The main contributions of this thesis are:

- A review of the state-of-the-art in synthetic workload generation, with a focus on the generation of Zipfian variates.
- A study of variate generation algorithms, with a focus on the generation of Zipfian variates.
- An implementation of a Zipfian variate generator in C++20, with a focus on performance and algorithmic simplicity
- An evaluation of the performance of the implemented Zipfian variate generator in comparison to existing solutions

The results show that existing solutions are generally accurate and fast, while one framework, FIO, is incorrect for larger sample sizes. One of the algorithms, *Rejection-Inversion*, is still one of the fastest and the most accurate for generating variates of the target distribution, though table methods could present a new alternative if further optimized. Generally implementations based in Java, with around 27 million variates per second, were between 1.4 and 1.8 times slower than their C/C++ counterparts, with around 44 million variates per second. As such, these implementations can still be recommended for synthetic workload generation. Furthermore accuracy analysis shows that the implementations found in common benchmark suites are quite accurate: the ratio between sampled frequencies and theoretical ones are quite close to 1 for sampel sizes 10 times the support size while maximal absolute error is around 10^{-3} . It was also found out that the sampled distributions converge quite fast, with 10 times the sample size being a good rule of thumb for the number of samples needed.

2 Background

Before proceeding with the approach, it's important to do the following:

1. define the Zipfian distribution
2. visually recognize such a distribution
3. discuss discrete variate generation
4. introduce error metrics used to compare different implementations of these generators

2.1 The Zipfian Distribution

This distributions has had ample impact in the areas of quantitative linguistics and information theory [5], as well as within computer science in the fields of storage systems and Relational Database Management System (RDBMS)[2]. It states that "(...) when a list when a list of measured values is sorted in decreasing order, the value of the n -th entry is often approximately inversely proportional to n " [6]. Mathematically, this is expressed by

$$\text{word frequency} \propto \frac{1}{\text{word rank}} \quad (2.1)$$

The form of interest for this research has an additional parameter α , yielding

$$\text{frequency} \propto \frac{1}{\text{rank}^\alpha} \quad (2.2)$$

Before continuing onto defining the probability more rigorously, some terms are defined more precisely:

- N - the support size of the distribution
- k - the rank of the variate
- $f_{theo}(k, N, \alpha)$ - the theoretical frequency of the k -th variate

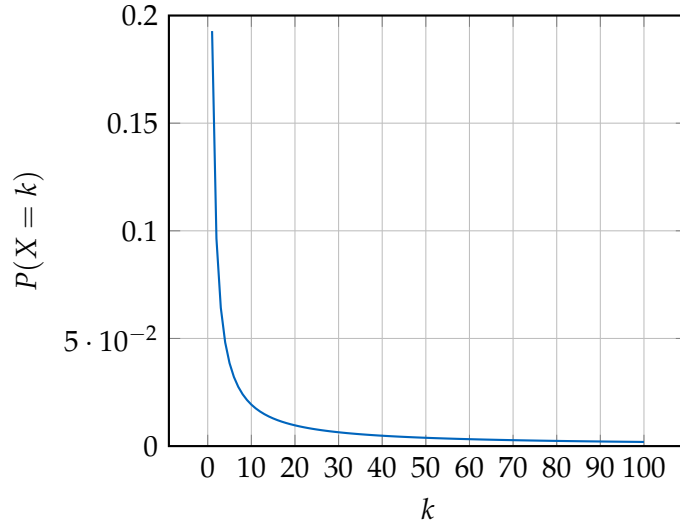


Figure 2.1: Probability Mass Function of a Zipfian distribution with $N = 100$ and $\alpha = 1$

With this, it's possible to write the following equation for the *PMF* of the Zipfian distribution:

$$f_{theo}(k, N, \alpha) := \begin{cases} \frac{1}{H_{N,\alpha}} \frac{1}{k^\alpha} & 1 \leq k \leq N \\ 0 & k < 1 \text{ or } N < k \end{cases} \quad (2.3)$$

where $H_{N,\alpha}$ is defined by the generalized harmonic number

$$H_{N,\alpha} = \sum_{k=1}^N \frac{1}{k^\alpha} \quad (2.4)$$

To give more concrete form to this distribution, we can visualize how the curve looks like for $N = 100$ and with an exponent $\alpha = 1$ in Figure 2.1.

2.2 Recognizing a Zipfian distribution

One way to visually recognize whether a sample follows a Zipfian distribution or not is by using a plot in log-log scale. This is justified by taking equation 2.3 (assuming the probability is always well-defined) and applying a log on both sides.

$$\log f(k, N, \alpha) = -\alpha \log k - \log(H_{N,\alpha}) \quad (2.5)$$

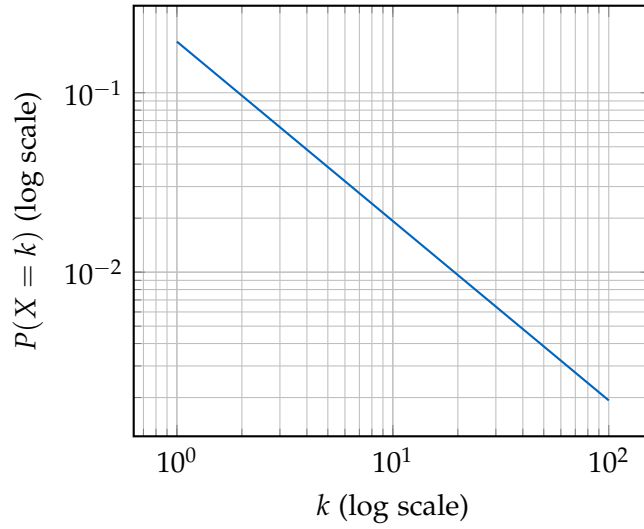


Figure 2.2: Application of log on data from 2.1

Since this is clearly a linear form: $Y = mX + C$, we should expect data that was generated according to a Zipfian distribution to have a linear form when transformed with the logarithm. Applying this on the last graph yields Figure 2.2.

It's a useful criterium for graphically evaluating if the Zipfian has the expected behaviour through the whole domain. It's not foolproof since other power laws also turn linear when transformed with the Zipfian, but since all generators analyse *try* to generate such a distribution, the problem disappears.

2.3 Discrete Variate Generation

Theoretically defining a distribution is one aspect of the work, but the main goal is to generate variates which follow a desired distribution. This can be represented by 2.6.

$$F(x) = Pr(X \leq x) \quad -\infty < x < \infty \quad (2.6)$$

The theory for developing variate generators rely on the following assumptions [7]:

- There exists a perfect uniform $(0, 1)$ generator that produces a sequence of *independent* random variables uniformly distributed in $(0, 1)$
- Computers can save and manipulate real numbers

Method	Efficiency	Complexity	Best Use Cases
Rejection Sampling [9]	Fast ($O(1)$)	Very simple and efficient	General-purpose, simple to implement
Rejection-Inversion Sampling [11]	Very fast ($O(1)$, best exact method)	More complex, efficient rejection	High-performance simulations, library implementations
Marsaglia's and Tsang's Condensed Table Lookup [12]	Very fast ($O(1)$, optimized for heavy tails)	Moderate, requires precomputed layer boundaries but low memory use	When many samples are needed efficiently, suitable for Zipf-Mandelbrot with infinite support
Jim Gray's Approximative Method [2]	Very fast ($O(1)$)	Doesn't require tables and is very fast but approximative (uses exponential approximation) for the tails	When many samples are needed efficiently and extreme tail precision is not necessary

Table 2.1: Summary of methods for sampling from the Zipf-Mandelbrot distribution

Which are directly violated once implementation in digital computers is considered. While not *a priori* a problem, since computers always introduce different types of errors in numerical computations, it's important to consider by *how much* these assumptions are violated. Even more so if certain distributions exacerbate problems by this violation [8].

There are many possible implementations and theoretical frameworks for creating discrete samplers, be them exact or approximative. Each method has its advantages and disadvantages and in the next sections, they will be evaluated. It's important to note, however, that other approaches are possible, but have been omitted since the ones present in the table 2.1 have already compared more favourably to them (see [9, 10, 7]). A brief discussion of the methods is presented in the next section.

2.3.1 Rejection Sampling

Rejection Sampling is an exact simulation method to generate observations from a distribution. It's working principle is quite simple: imagine plotting the probability

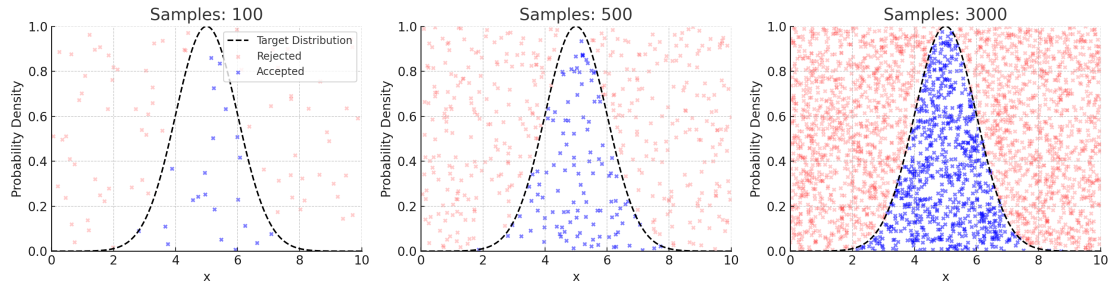


Figure 2.3: To generate these graphs, a random uniform sample is taken for the x-axis as well as another normalized one for the y-axis.

density function of the distribution to a large rectangular board. Onto this board, darts are thrown at random (assuming every position is equally as likely) and the darts that land under the area of the curve are kept. After throwing enough darts, a clear pattern starts to form that resembles the distribution [13]. The graphs in Figure 2.3 illustrate this principle well - blue points are accepted whereas red points are rejected using the previously described criterium.

It's important to notice, however, that for this method to work, a *bounding function* is needed. In the case of the Zipf, this could be simply $1/k^{-\alpha}$, which is easier to calculate than the true Zipfian distribution. While the bounding function must be comparable to the original function, it must only *be greater up to a constant factor* - this eliminates the need for the generalized harmonic number, which is computationally expensive to calculate. A disadvantage of this method is that two random samples are required: one for the x-axis, another one for the y-axis.

2.3.2 Rejection-Inversion Sampling

Rejection-Inversion Sampling due to Hörmann and Derflinger [11] is a more efficient version of the Rejection Sampling method. It's based on the following observations regarding the naive rejection method:

- A naive rejection algorithm requires two samples to generate the target variate.
- The Zipfian is bounded by a monotonically decreasing and convex function, which makes it possible to invert the bounding function.

These two observations lead to the following principle:

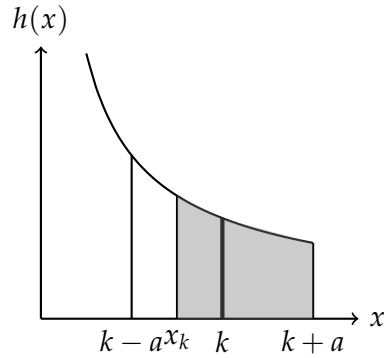


Figure 2.4: Rejection-Inversion Sampling for a Zipfian distribution

Generate a random variate u from the uniform distribution $U(0, 1)$ and invert the bounding function to find the corresponding $x = H^{-1}(u)$. Define a bounding area around the nearest integer k (with radius a). If $x \geq x_k$ and $x \leq k + a$, accept X as a sample. Otherwise, reject it.¹

To visualize the principle used, consider figure 2.4: after k was found using the inversion method, the bounding area is defined for radius a . This creates an interval of $k - a$ and $k + a$, which is then used to accept or reject the sample x . Typically, a is chosen to be $1/2$ [11].

2.3.3 Marsaglia's and Tsang's Condensed Table Lookup

To generate a variate from any discrete distribution, it's possible to populate with as many occurrences of a given key as it occurs in the frequency table and then use a uniform sample to find the corresponding variate. This is called a table method [12]. As an example, consider figure 2.5: One takes a sample from an uniform Pseudo-Random Number Generator (PRNG) and uses it to access an index in the array.

This approach has a big limitation: the size of the array is $O(n)$, which is not feasible for large support n . To circumvent this, Marsaglia and Tsang proposed a *condensed table lookup method* [12], which is based on the following principle:

¹This is a simplified version of the algorithm. For a more detailed explanation, see [11]

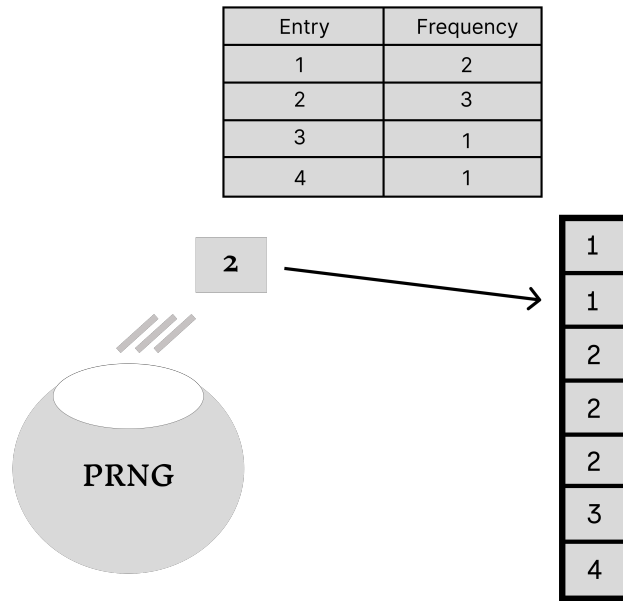


Figure 2.5: The table contains the frequency of each index, whereas the array is populated with as many entries as there occurrences of each index. The sample "2" was generated, which is used to access the 2nd position in the array

A probability is represented by a floating-point number p with d decimal places. For example, assuming $d = 2$, the entry 1 from the last figure has probability $p(1) = 0.28$. To realize this probability, a table of size 10^2 is required. The trick to reduce space usage is to break this table down onto d tables instead of using one big table with 100 entries. To do this, each probability gets split into d digits m , which respectively get m entries in the corresponding table. Now, generating a uniform integer from the interval $[1, 100]$, it's possible to access the correct table and generate the variate.²

2.3.4 Jim Gray's Approximative Method

Jim Gray's method [2] is a fast approximative way to generate samples from a Zipfian distribution. It's based on an approximation of the generalized harmonic number using integration for the first most frequent elements and a exponential approximation for

²This is a simplified version of the algorithm. For a more detailed explanation, see [12]

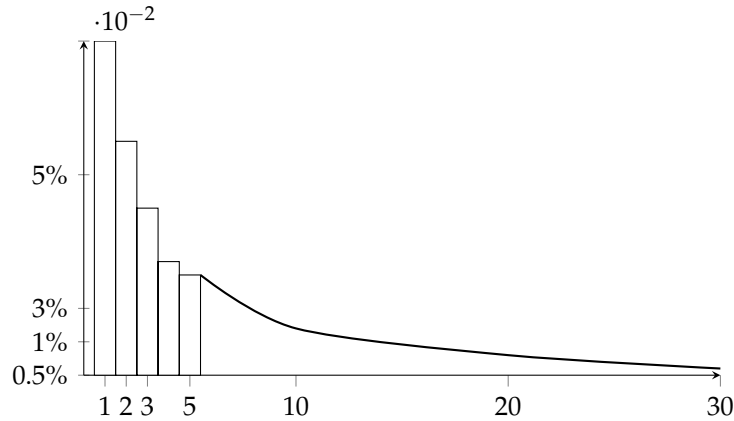


Figure 2.6: Jim Gray's method for generating samples from a Zipfian distribution

the tail. A uniform sample is then mapped onto an index using inversion. Figure 2.6 makes this idea clear by having two different areas - the first uses the precise discrete values, whereas the second uses an exponential approximation.

The mathematical description of this method is not quite straightforward and is omitted here. For a more detailed explanation, see [2, 14].

2.4 Error Metrics

To begin comparing the different methods introduced previously, it's necessary to first define the framework used to evaluate them. Drawing from the literature [15, 16, 17], the following principles are used:

1. Calculate the theoretical distribution for the given parameters: N - support size, and α - skew
2. Generate a empirical distribution with the developed tool as explained in chapter 3
3. Use a **goodness-of-fit test** to compare the generated distribution to the theoretical one

This framework assumes a "black-box" view of the generating processes and treat the generated samples as a dataset that should be fit to the distribution we presume it was sampled from - in our case, the Zipfian. At the heart of this comparison is

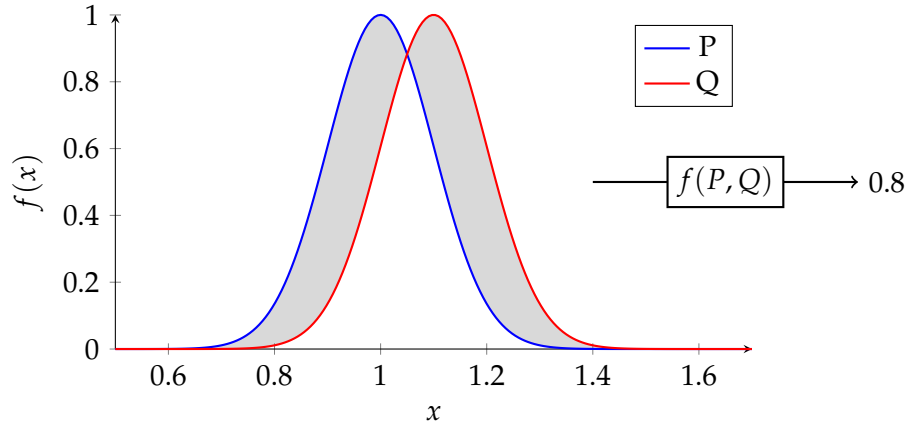


Figure 2.7: Accuracy Measure

a **goodness-of-fit test**, which tells whether the empirical distribution matches the expected distribution closely enough. This requires a comparison metric between the two distributions, which can be thought of as a function that takes two distributions and returns a number that quantifies the difference between them. This idea can be made more concrete by looking at the example given in 2.7. If the blue curve represents the theoretical distribution and the orange curve the empirical one, the shaded area represents the difference between the two. The smaller this area, the better the empirical distribution fits the theoretical one.

For the purpose of this work, two measures are going to be used: the **Kolmogorov-Smirnov** distance and the **Total Variation** distance [18, 16]. The former is used in the cited literature previously and has the advantage of being a non-parametric test, which means it doesn't require any assumptions about the underlying distribution. The latter is a more general measure of the difference between two probability distributions and is more resistant against outliers.

Kolmogorov-Smirnov Test While there is much theory behind the Kolmogorov-Smirnov test, the good-of-fit measure is simply the maximum difference - $D_n(P, Q)$ - between the empirical and theoretical distributions. A formal definition can be found in Equation 2.7. It can be understood best with the graph in Figure 2.8. This distance measure is compared with a theoretical distribution evaluated for the same parameters (see Equation 2.8).

$$D_n = \sup_x |P(x) - Q(x)| \quad (2.7)$$

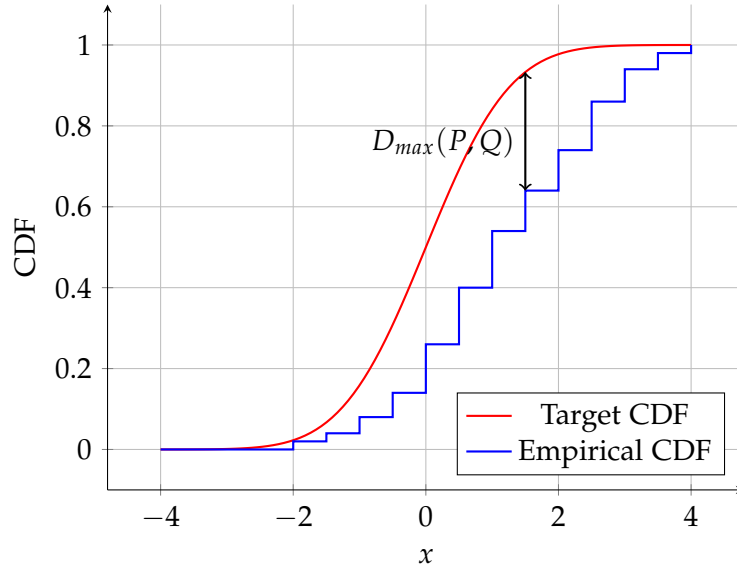


Figure 2.8: Kolmogorov-Smirnov Test

As explained before, a theoretical distribution is associated with this measure so as to check whether the two distributions truly match.

$$F_n = P[D_n \leq x | \mathcal{H}_0] \quad \text{for } x \in [0, 1] \quad (2.8)$$

As this function is quite hard to accurately calculate and approximations are used for big sample sizes. The treatment given is due to Marsaglia (see [18]) and is the choice of implementation for the *SciPy* library [19] used to generate the metric.

Total Variation Distance The total variation distance is defined as the sum of all absolute differences in the probability mass function of two distributions. More precisely, given two (discrete) distributions P and Q defined in X

$$d_{TV}(P, Q) = \frac{1}{2} \sum_{x \in X} |P(x) - Q(x)| \quad (2.9)$$

It ranges from 0 to 1, where 0 means the distributions are equal and 1 means they are completely different. Interpreting it as a percentage allows statements as: "About $y\%$ of the probability mass differs between the theoretical and empirical distribution". To better understand how this measure looks visually, observe that this measure would be equivalent to the shaded area in figure 2.7.

An important property of the total variation distance is that with growing sample sizes, the distance between the empirical and theoretical distribution should converge

to 0 for all x [20]. Associated to this metric is the *Dvoretzky-Kiefer-Wolfowitz (DKW) inequality* [21], which gives a bound on the probability that the distance between the empirical and theoretical distribution is greater than a certain value.

Why the tests are not perfect While these tests are widely used in the literature, they are not perfect. This work stops short of making hypothesis test due to the bounding conditions of the rejection criteria used. Both metric tests converge fast to 0 with growing sample sizes, which makes rejecting the null hypothesis very hard. Therefore, the analysis will limit itself to presenting which methods are *more correct* than others and how *wrong* they are in absolute terms. For the purpose of this work this approach suffices, as it can be determined how inaccurate latency measurements will be compared to the real distribution and estimates can be made on how much this will affect the system, e.g. *if the probability of tail indexes is off by 10%, latency measures will be off by 5%*. Additionally, other techniques will be used to ensure the correctness of the Zipfian implementations.

3 Approach

The goal of this thesis is to identify the most widely used frameworks for generating zipfian synthetic data in the systems programming and database community and make a comparative evaluation. The first step required to achieve this is a systematic review of current approaches. The following sections describe

1. the **research approach** used to identify the frameworks,
2. the **common frameworks** recognized from the search along with short descriptions
3. the **popularity and impact** of those tools
4. the **benchmarking module** which will be used to evaluate the frameworks

3.1 Research Approach

To identify relevant frameworks used by the systems programming and database community, first the scope of venues and journals was defined. The list is comprised of:

- Proceedings of the VLDB Endowment (PVLDB)
- ACM Special Interest Group on Management of Data (SIGMOD)
- Conference on Innovative Data Systems Research (CIDR)
- USENIX Symposium on Operating Systems Design and Implementation (OSDI)
- USENIX Symposium on Networked Systems Design and Implementation (NSDI)
- USENIX Conference on File and Storage Technologies (FAST)

These conferences were chosen due to their high selectivity and special relevancy for storage technology research, where there is a prevalence of standardized IO benchmarks. The search was conducted broadly following the steps indicated in [22], as defined below:

1. Word search on keyword search matrix (see table 3.1)
2. Identify the frameworks used in the papers
3. Extended keyword search, this time adding the frameworks identified in the previous step

The first step suggests the use of a keyword matrix. This works by choosing appropriate keywords which refer to the topic of interest. Then for each such keyword, synonyms are used to improve the accuracy and scope of the search. It's defined below as:

	Keyword 1	Keyword 2	Keyword 3
synonyms	zipf zipfian	benchmark workload evaluation empirical dataset	synthetic generated sampled sampler

Table 3.1: Keyword search matrix

The columns are the keywords which are boolean OR'ed, whereas the rows are boolean AND'ed. This means that at least one word of each column should appear in the search.

These queries were then run for the conferences and journals listed before on an unrestricted timescale. The databases consulted are

- IEEE Xplore
- ACM Digital Library
- DBLP
- Google Scholar

3.2 Common Frameworks

On a first phase, the following frameworks were identified among the top 20 results of each conference

Frameworks
Yahoo! Cloud Serving Benchmark (YCSB) [23]
Flexible I/O Tester (FIO) [24]
Sysbench [25]
OLTP-Bench [26]

These frameworks were then searched with the same keyword matrix and their name as a last keyword. This has yielded the following results:

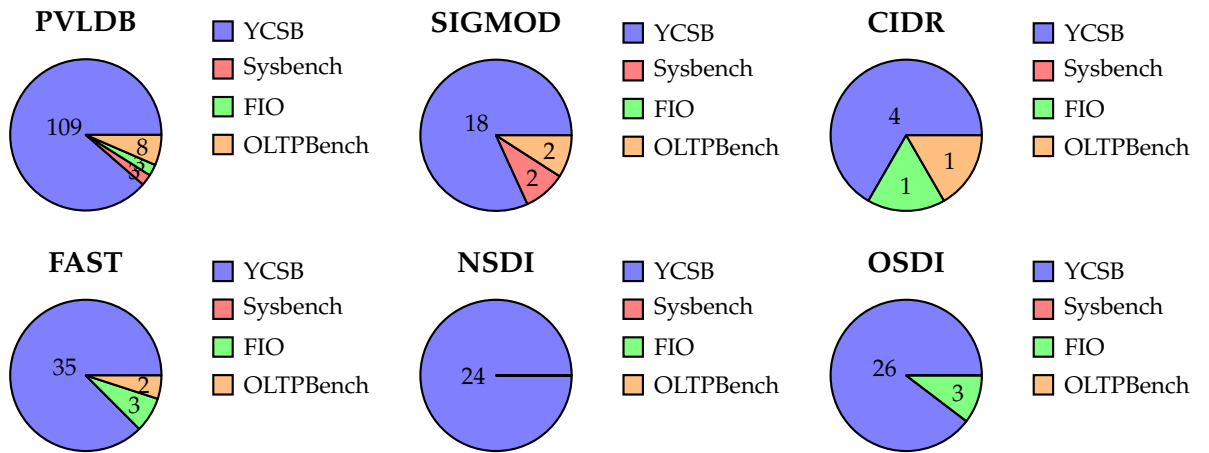


Figure 3.1: Pie charts for each conference (excluding Web Polygraph and pgBench).

The most popular framework by far is **YCSB**, then followed by **Sysbench** along with **FIO**. Other frameworks are not as prevalent, but have appeared in the search results.

3.2.1 Yahoo! Cloud Serving Benchmark

YCSB is a widely used benchmarking tool for cloud databases. It was developed by *Yahoo! Research* and is now maintained by a community of developers. Its working principle is to generate workloads in a threaded environment, such that the databases can be also tested for concurrent accesses. It's also capable to loading tables with data organized according to different distributions, among which the Zipfian distribution.

To achieve this, YCSB uses a Zipfian generator which is based on the one proposed by Jim Gray (see 2.3.4). One limiting factor on his original paper is that it's the generated keys are lexicographically sorted, meaning the first rank (in the case of an integer, 0) is the most frequent. To better reflect real workloads, the keys should be randomly distributed within the range. See figure 3.2 for a comparison of the these two cases.

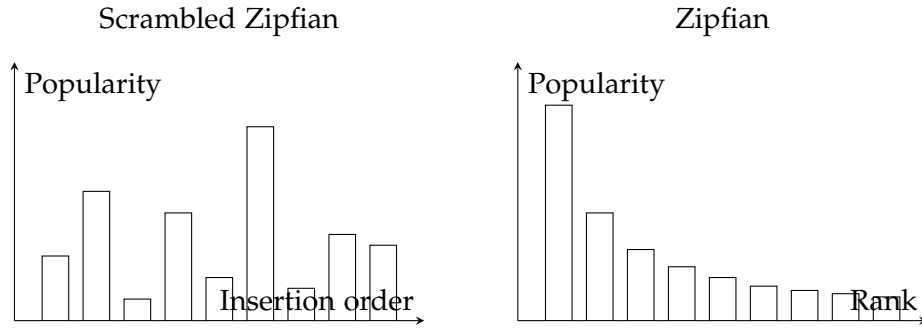


Figure 3.2: Comparison of Jim Gray's version (right) and the one employed by YCSB (left).

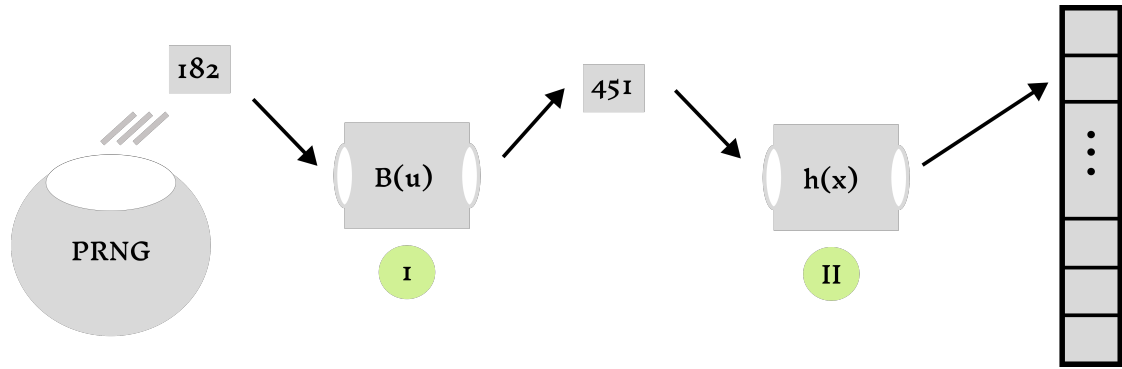


Figure 3.3: YCSB does sampling by inversion (1) and then hashes the value to spread in the available range (2)

To go around this issue, YCSB uses a hashing algorithm to randomly spread the values in the range as well as employing an extended domain to reduce collisions [23].

3.2.2 Flexible I/O Tester

FIO [24] is a benchmarking tool for storage systems. Much like YCSB, its purpose is generating workloads following a certain pattern access to test a storage system. The workloads can vary in type, including sequential or random and read-only, write-only or mixed. In the case of **random I/O**, the user is able to change the access pattern with the goal of simulating both asymmetrical and more symmetrical workloads. The underlying distributions include the Zipfian, Normal, Pareto, Uniform distributions.

The generation of a Zipfian distribution works by the same principle as YCSB: first sample a variate using the approximative method and then hash it to spread it in the

available range. As was the case before, the idea from 3.3 also applies here.

3.2.3 Sysbench

Sysbench is a benchmarking tool for databases as well as storages systems. It works in a manner very similar to YCSB, using multi-threading to strain the system with concurrent accesses. The tool is capable of generating synthetic data according to different probability distributions, among which the Zipfian distribution.

What sets it apart from the previous ones is that the variate generation uses a different algorithm. The one employed by Sysbench is based on the *rejection-inversion principle* (see 2.3.2) and the implementation is based off the Apache Commons Math library [27], under the name *RejectionInversionZipfSampler*.

3.2.4 OLTP-Bench

The OLTP-Bench [26] strives to unify different types of benchmarks for evaluating a given Database Management System (DBMS). It favours a modular design, enabling very different benchmarks to be used under a common interface, which makes it quite extensible. However, it also has enough benchmarks built-in to be used directly, among which AuctionMark [28], Transaction Processing Performance Council Benchmark C (TPC-C) and YCSB. The focus will be rather layed upon the AuctionMark benchmark, as it's the one excepting YCSB which generates synthetic data following a Zipfian distribution. The implementation is almost directly based off of the one from YCSB, and therefore in further analysis, it will be considered as such directly.

3.3 Benchmarking Module

To evaluate the frameworks, a benchmarking module was developed. It was written in Python and allows the user to specify which sample generator to use, as well as the distribution parameters. It's usable from the command line and includes the following commands: `sample_zipf`, to generate a Zipfian distribution, `accuracy_zipf` to calculate accuracy measures for one distribution, and lastly `benchmark` to run a specified performance benchmark. It follows a modular design, with the following interfaces: *SamplerInterface*, responsible for providing a common wrapper interface to all the sampling algorithms, *StorageInterface*, which defines how the data is to be stored in-memory before processing, and *OutputInterface*, which defines how the data is to be persisted. A UML diagram of the tool is shown in figure 3.4.

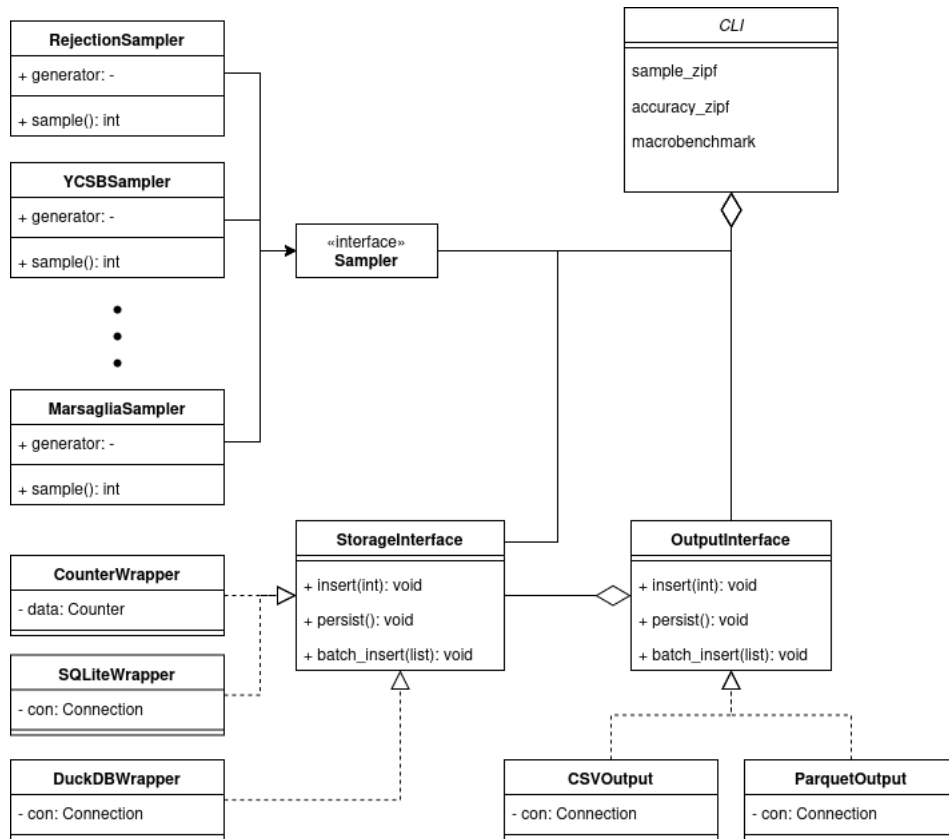


Figure 3.4: UML diagram of the benchmarking module

3.3.1 Commands

To generate a sample, the user must use the command `sample_zipf`, such as

```
sample_zipf --generator sysbench --n 1000 --samples 1000 --skew 1.5 --  
output csv --storage sqlite --buckets 100
```

This will yield a CSV file with 1000 samples following a Zipfian distribution with skew 1.5 and domain from 0 to 1000. The samples will be stored in-memory in a SQLite database with 100 buckets.

In order to calculate the accuracy measures of the distribution implied by a sample stored in a CSV, the user must use the command `accuracy_zipf`, such as

```
accuracy_zipf --input_dir /path/to/csv --skew 1.5 --output_dir /path/to/  
output
```

This method assumes that the directory contains CSV files of results of various sample sizes and domain sizes, but belonging to one generator. It will then generate 3 csv files, with columns for domain size and rows for domains sizes - the first one with the Total Variation Distance (TVD), the second with the Kolmogorov-Smirnov (KS) distance and the third with the p-value the KS test.

4 Evaluation

In the previous chapters, some variate generators included in common frameworks were briefly presented, as well as the underlying algorithms used to sample the Zipfian distribution. To make a fair comparison, it's necessary to evaluate the accuracy of the distributions generated, as well as the performance of the algorithms used to generate them. Additionally, simplicity of the methods will be considered.

- **Accuracy:** The statistical properties of the generated distributions will be compared to the expected values given by the theoretical Zipfian distribution
- **Performance:** The time taken to generate the distributions will be measured and compared using standard profiling tools as well as custom-written benchmarks

The variate generators considered for this approach are exactly those presented in the previous chapters: YCSB, FIO, Sysbench and OLTP-Bench (which will be aliased by Rejection-Inversion - Java), along with implementations of the algorithms discussed in 3: Rejection and Condensed Table Lookup. Rejection-Inversion wasn't separately implemented since Sysbench already does it.

4.1 Experimental Setup

This section describes the hardware and software environment used to conduct the experiments. All experiments described in this chapter were conducted on a machine with the specifications detailed in table 4.1. The software environment used is detailed in table 4.2. Since the samplers were implemented in different languages, it's also necessary to detail the versions of the interpreters used to run them (see 4.3).

4.2 Methodology

Evaluations were conducted both for accuracy and performance.

CPU Model	Clockspeed	Number of Cores	Memory Capacity	Memory Speed
AMD Ryzen 7 5850U	3.6 GHz	8	16 GB	3200 MHz

Table 4.1: Computer Specifications

Operating System	Kernel Version	Relevant Tools
Debian GNU/Linux 12	6.1.0-amd64	perf[29], JMH [30], Google Benchmark [31]

Table 4.2: Software Specification

Language/Version	Compiler	Benchmark/Sampler
C++20	gcc 12.2.0	Own implementation
Java 17	OpenJDK 17.0.8	YCSB
C (GNU17)	gcc 12.2.0	Sysbench, FIO
Rust	1.58.0	None

Table 4.3: Interpreter Versions

Accuracy To measure accuracy 2 quantifiable criteria will be used to offer well-rounded view of the quality of the generated distributions - the **Kolmogorov-Smirnov test** and the **Total Variation Distance** as explained previously in 2. All of these are well explored in the literature [15, 16, 32, 17].

Besides these numerical quantities, a qualitative approach will be also taken to ensure the measures can be well interpreted. To this end, two visual techniques will also be used: a **log-scaled graph** (see 2.2) and a **frequency ratio analysis**. These help identify systematic deviations as well as deviations correlated with the rank of the indexes. Table 4.4 gives an overview of the criteria used for the evaluation and how each simulation was run.

Performance The main measure of interest is *throughput* - the amount of variates generated in a time interval (e.g. $10M/ms$). Since benchmarking in modern systems is a rife with pitfalls [30, 33], some precautions should be taken before running the code. More details follow in the coming sections.

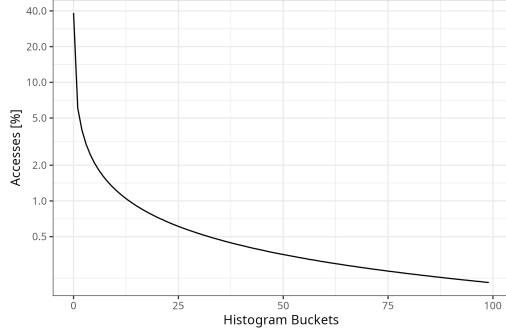
Criteria	Support Size	Skew	Number of Samples
Log-scaled graph	10^6	0.8	10^9
Frequency Ratio Analysis	10^6	0.8	10^9
Kolmogorov-Smirnov Test	10^8	0.8	10^4 to 10^9
Total Variation Distance	10^8	0.8	10^4 to 10^9

Table 4.4: Simulation specification for each accuracy criteria

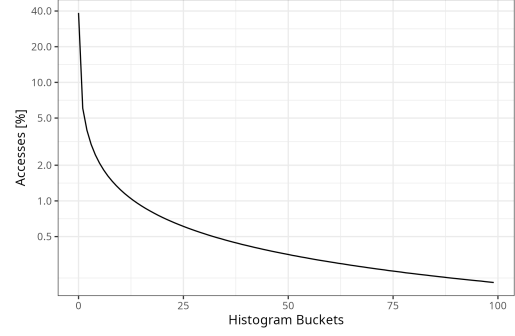
4.3 Evaluating accuracy

To first identify which distributions are very wrong, a simple visual inspection is done of the graphs in Figure 4.1. The only distributions which visibly deviate from the expected curve are the FIO and the Condensed Table Lookup. These will be investigated later on.

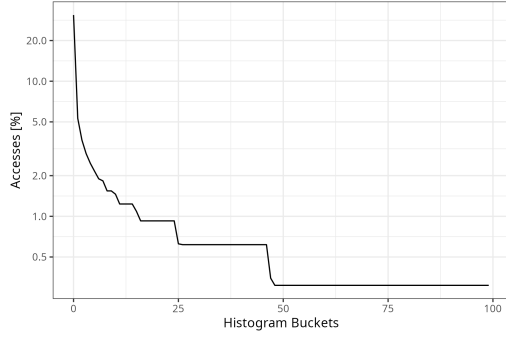
4 Evaluation



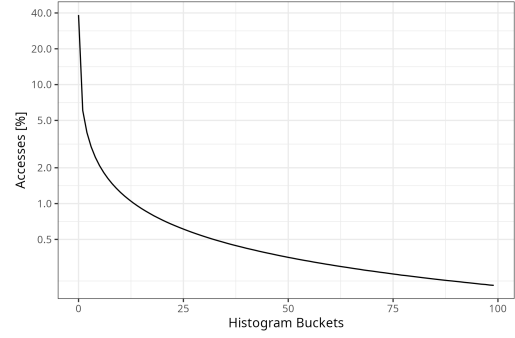
(a) Rejection inversion



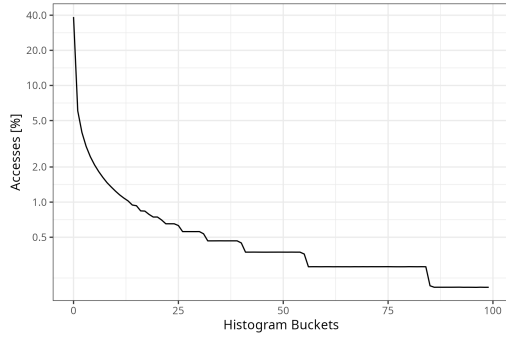
(b) YCSB



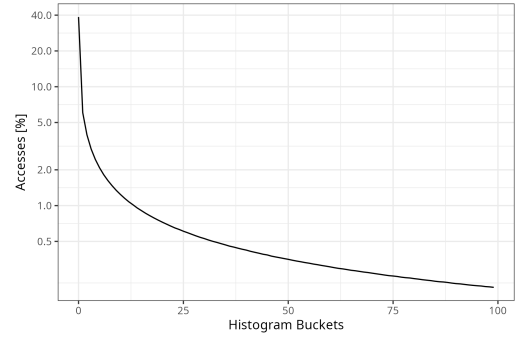
(c) FIO



(d) Sysbench



(e) Condensed Table Lookup



(f) Rejection

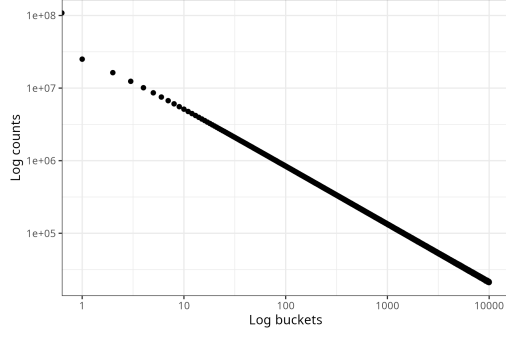
Figure 4.1: All distributions have support size 10^8 and have 10^9 samples

Log-scaled graphs These graphs allows for identifaction of systematic biases in data generation (as explained in Chapter 2). The y-axis represents the *counts* and the x-axis the *histogram entries* (or "buckets") for all ranks, as shown in figure 4.2. As was the case before, only FIO and the condensed table lookup implementations have glaring deviations from the expected straight line.

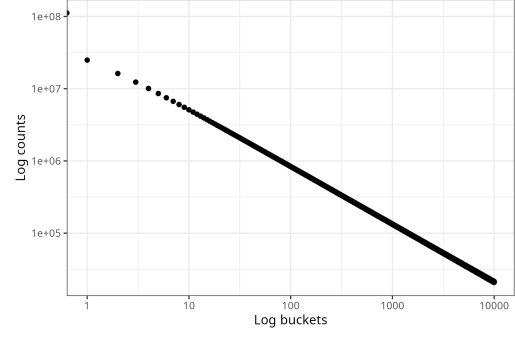
Frequency Ratio Analysis To further analyse how the accuracy varies within the domain of the distribution, a frequency ratio analysis is conducted. This is done by plotting the ratio of the empirical frequency to the theoretical frequency (see Equation 2.3) against the rank of the variate - $\frac{f_{emp}(k)}{f_{theo}(k)}$, where k denotes the rank.

The graphs in figure 4.3 show the results for the 6 variate generators. Clearly, the more problematic distribution is once again FIO, while other implementations fluctuate around 1.0 - the expected value. Observing the equation 2.3 and the graph of *FIO* in figure ?? dismysifies the problem: The probability stays constants for long stretches of the domain, whereas the real distribution ever so decreases. This in turn makes the ratio of the empirical to the theoretical frequency increase, before falling off again.

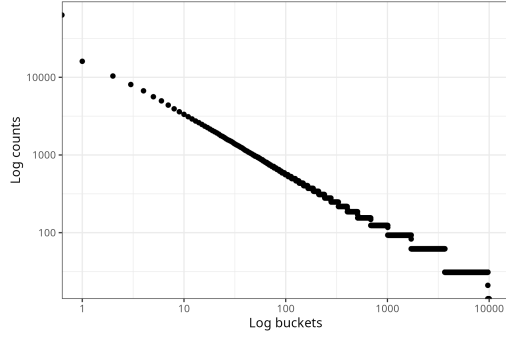
Quantitative Criteria First follows the **Kolmogorov-Smirnov test** in Figure 4.4 and the **total variation distance** in Figure 4.5. From both graphs, we notice that the FIO implementation is the one that deviates the most from the expected values, while the other implementations are very close to the expected values. Since all but FIO converge to smaller error values, we can discard the hypothesis that some there are systematic errors and that all implementations are accurate. While there is some evidence that moments (such as mean, variance) of self-similar distributions, such as the Zipfian distribution, only very slowly converge to their theoretical values in a sampling environment [34] , it seems that for the *Rejection Inversion* (Sysbench, AuctionMark), *Rejection* and *approximative* (YCSB, FIO) sampling algorithms, the distributions are very close to the expected values given enough samples. The table 4.5 summarizes the results for sample size = 10^9 .



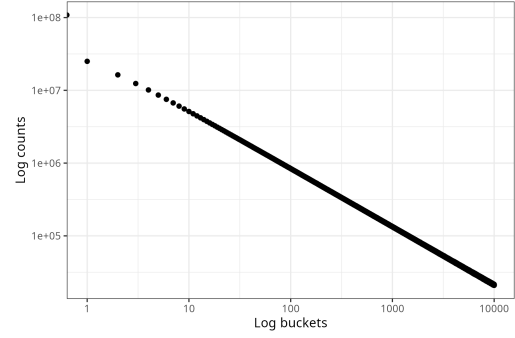
(a) Rejection-Inversion



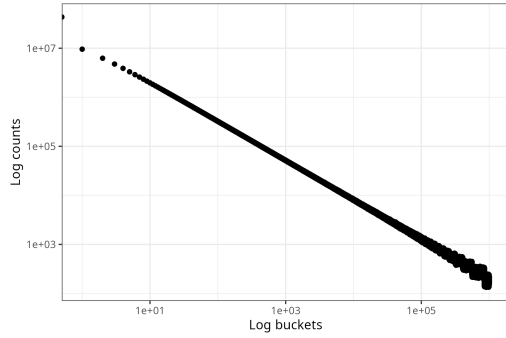
(b) YCSB



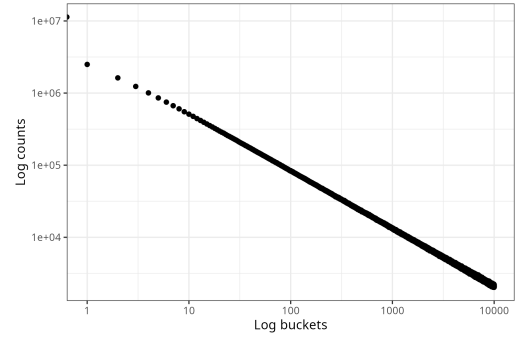
(c) FIO



(d) Sysbench

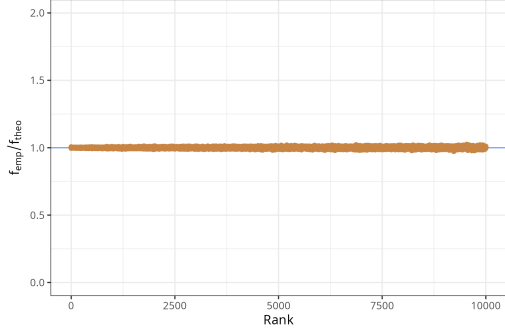


(e) Condensed Table Lookup

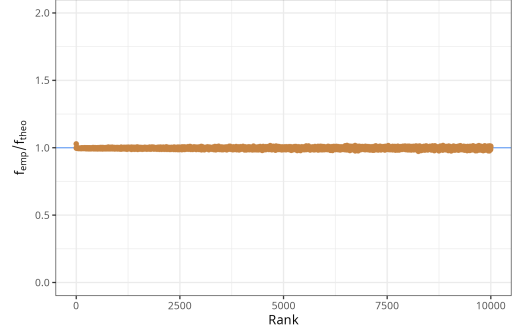


(f) Rejection Sampling

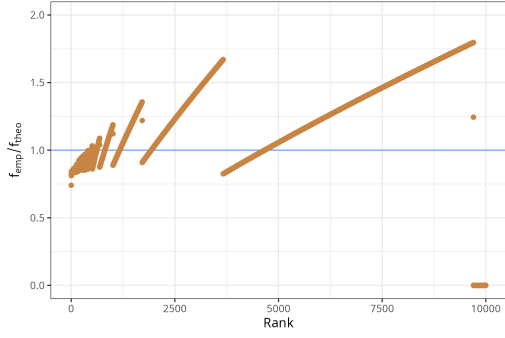
Figure 4.2: Log-log plots of the performance of the 6 different variate generators.



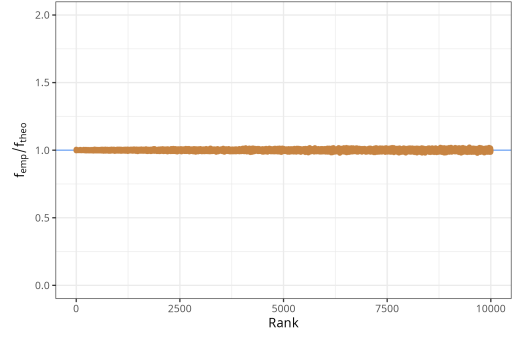
(a) Rejection inversion



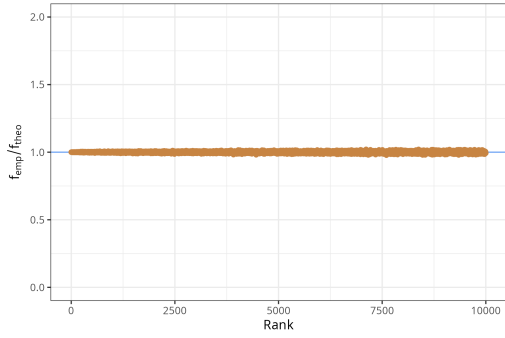
(b) YCSB



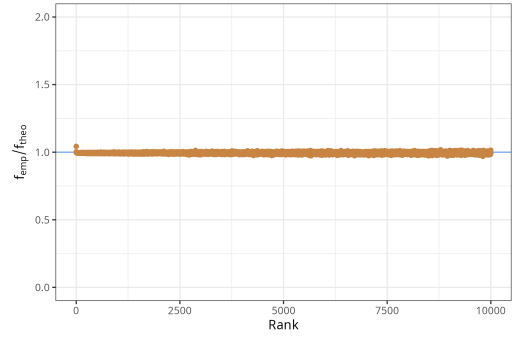
(c) FIO



(d) Sysbench



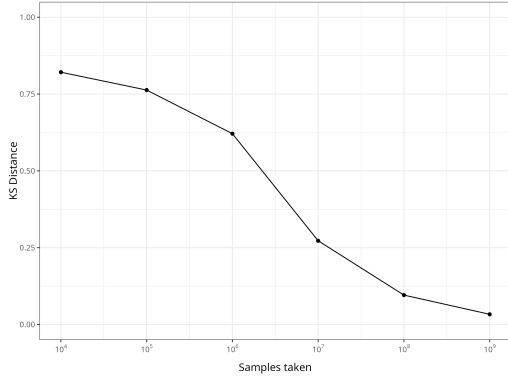
(e) Condensed Table Lookup



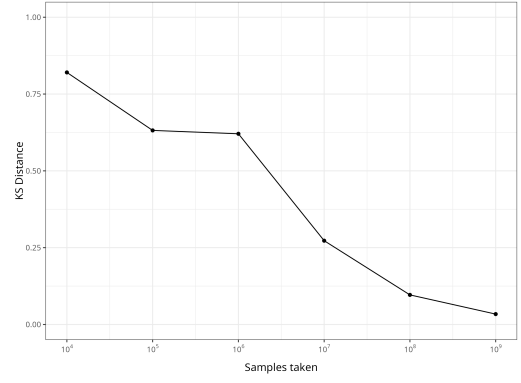
(f) Rejection

Figure 4.3: All distributions have support size 10^6 and have 10^9 samples

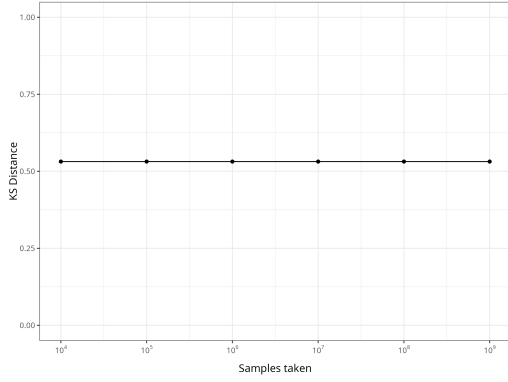
4 Evaluation



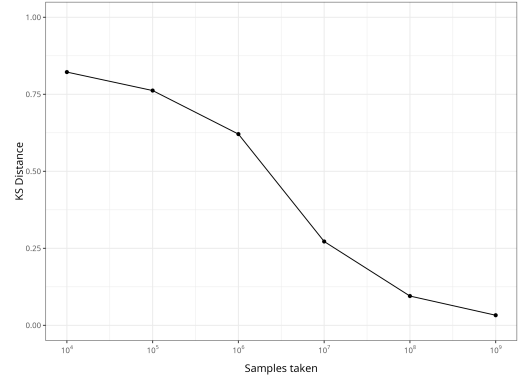
(a) Rejection-Inversion



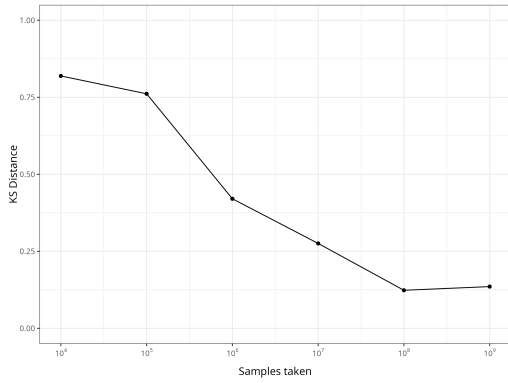
(b) YCSB



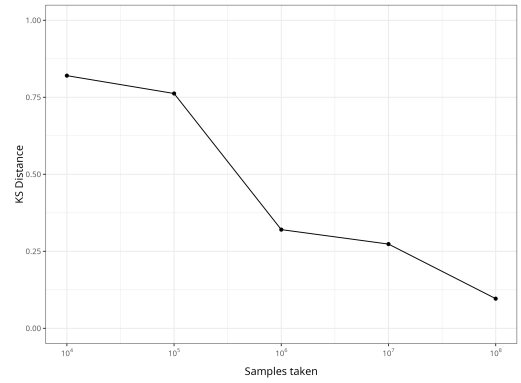
(c) FIO



(d) Sysbench



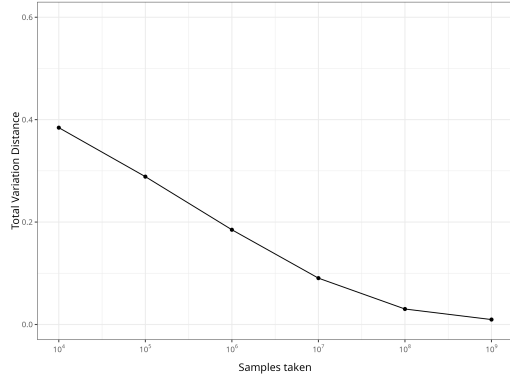
(e) Condensed Table Lookup



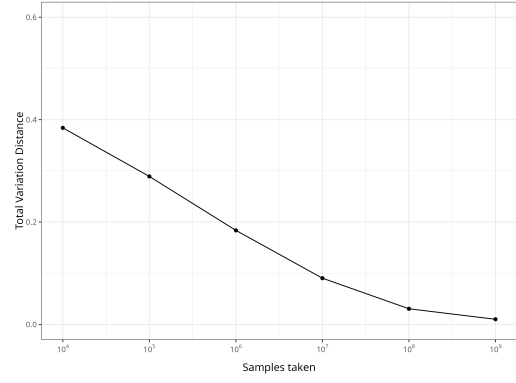
(f) Rejection Sampling

Figure 4.4: Kolmogorov-Smirnov test results for the different samplers.

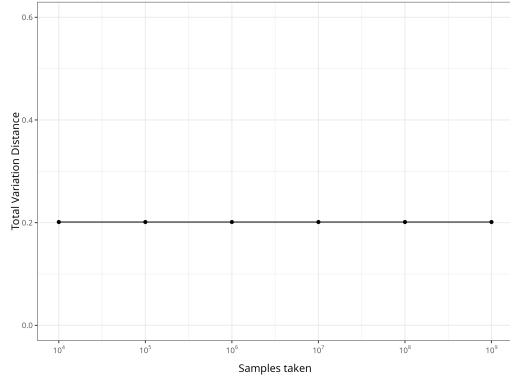
4 Evaluation



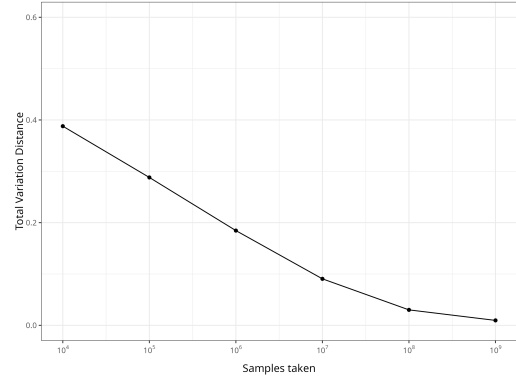
(a) Rejection-Inversion



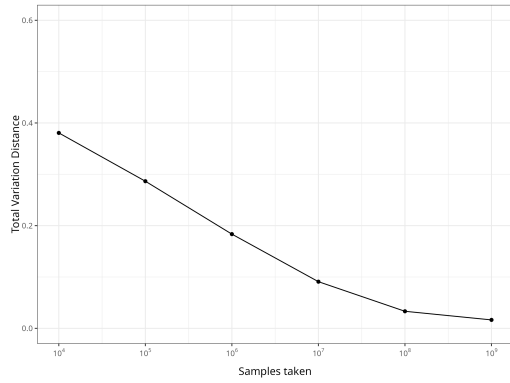
(b) YCSB



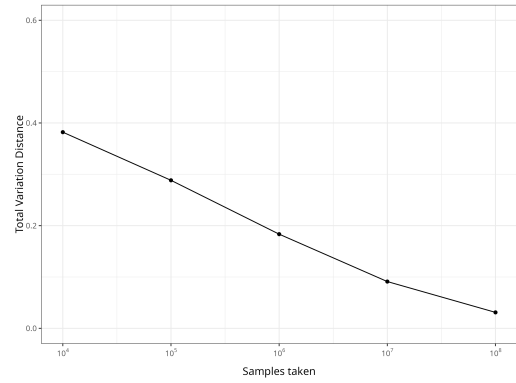
(c) FIO



(d) Sysbench



(e) Condensed Table Lookup



(f) Rejection Sampling

Figure 4.5: TVD test results for the different samplers.

Sampler	Total Variation Distance	Kolmogorov-Smirnov Statistic
FIO (C)	0.201	0.532
Rejection-Inversion (Java)	0.00963	0.0332
YCSB (Java)	0.0101	0.0338
Sysbench (C)	0.00963	0.0328
Condensed Table Lookup (C++)	0.0164	0.135
Rejection (C++)	0.031	0.0963

Table 4.5: Accuracy Results for 10^9 samples

Analysis of Accuracy All implementations but FIO and the condensed table lookup present very accurate tendencies when the sample size increases to the scales utilized in benchmarking, which confirm that they are safe to use. It now remains to explain why both referenced algorithms are not quite as accurate.

The FIO implementation likely presents systematic deviations from the expected values due to the hashing function used to preprocess the variates, as the variate generation itself follows exactly Jim Gray's pseudocode. The choice of hashing function falls under the umbrella term "Jenkins Hash Function" (more specifically *lookup3*, see [35, 36]) to remap all generated variates before they are counted. Doing this, however, introduces significant distortions in the distribution, since the aforementioned hash function is not perfect. Furthermore a good hash function should be mostly uniform, the effect of which can be seen in the graph above as long intervals of constant probability occur. The drops are explained by the fact that eventually the lower-rank values eventually "differ" enough their bit representation so as to be hashed to a different value.

The case for the condensed table lookup is more complex. The algorithm is based on a table lookup with precision limited to 2^{30} , or rather $5 \cdot 10^9$. This means that the algorithm is not able to generate probabilities which only differ by a factor of 10^{-9} will be mapped to the same value. Testing for equality after this mapping yields that the probabilities are 99.9 pairwise equal among all ranks, which is more than sufficient to account for the long stretches of constant probability seen in Figure 4.1. This effects gets amplified the lower the rank, which explains the ever stretching intervals of constant probability. A possible improvement would be to increase the precision of the table lookup, but this would also increase the memory requirements of the algorithm and make it closer to the naive approach.

4.4 Evaluating Performance

As already mentioned, evaluating performance is a tricky business. The main measure of interest is *throughput*, which is the amount of variates generated in a time interval. What makes it complicated is controlling for external factors outside of the segment of code being tested. To this end, microbenchmarking is used and common techniques are applied to avoid pitfalls [30, 33].

4.4.1 Microbenchmarking

Microbenchmarking is the process of measuring the performance of small code snippets, usually in the order of microseconds. The main goal is to measure the performance of the code in isolation, without the influence of other factors. Since the languages of interest are primarily C++, C and Java (see table 4.3), the tools used to measure performance will be Google Benchmark [31] for C++ and C, and JMH [30] for Java. For C++ and C applications, the following precautions will be taken (for instruction on how to accomplish the following, see [37]):

- Set specific CPU core for the process
- Disable frequency scaling
- Disable turbo boost
- Limit the available memory to the process
- Warm-up runs

At last, it's important to define the runs. Since what is going to be measured is throughput, we define the duration of a single run to be 1 second. This will be repeated 30 times and the average throughput will be calculated. Since the speed is invariant to domain size and distribution parameter, these are kept fixed at 10^8 and 0.8, respectively.

For the Google Benchmark, the following command will be used to run the benchmarks (for more details on how to use the tool, see [31]):

```
./benchmark --benchmark_repetitions=30 --benchmark_min_warmup_time=1.0
```

As a precaution for the sample generation not be optimized away, the function `benchmark::DoNotOptimize` on the results of the computation was used. Similarly for the JMH (more details on [30, 38]):

```
java -jar target/benchmarks.jar -bm thrpt -wi 10 -i 30 -f 1 -r 1
```

Sampler	Average Throughput (M/s)	Standard Deviation
FIO (C)	26.2	0.01
Rejection-Inversion (Java)	27.4	0.07
YCSB (Java)	29.9	0.14
Sysbench (C)	43.6	0.15
Condensed Table Lookup (C++)	45.2	0.06
Rejection (C++)	28.1	0.3

Table 4.6: Microbenchmarking Results

The results depend heavily on which JVM is used, but some common patterns exist to avoid making code be optimized away:

- Use the `Blackhole` object to consume the results of the computation
- Use the `@CompilerControl` annotation to avoid inlining
- Use the `@State` annotation to avoid dead code elimination

From the table 4.6, we can recognize that the Condensed Table Lookup sampler is the fastest, which is to be expected given that the main operation is a table lookup, while the cost gets offloaded to the initialization. The Sysbench sampler is the second fastest, which is expected given the overhead of the JVM. But to further investigate why this difference is present, arun with a profiler is required. Running `perf`[29] along with the JMH tool yields the following results for YCSB and Rejection-Inversion (Java). Only relevant lines are presented.

Rejection-Inversion (Java)

```
Baseline: 565.140.532.065 instructions 4,33 insn per cycle
Sampler: 204.065.670.512 instructions 1,56 insn per cycle
Sampler: 11.310.090 L1-dcache-load-misses 0,02% of all L1-dcache
accesses
```

YCSB

```
Baseline: 578.884.719.478 instructions 4,43 insn per cycle
Sampler : 392.403.334.414 instructions 2,92 insn per cycle
Sampler: 339.375.866 L1-dcache-load-misses 0,43 % of all L1-dcache
accesses
Sampler: 789.273 L1-icache-load-misses 0,30% of all L1-icache
accesses
```


Given that both the baseline and the actual sample routine have similarly high IPC and low number of data and instruction cache misses, the lower throughput in Java is likely not caused by an inefficient implementation, but rather by the overhead inherent to the JVM runtime, such as garbage collection, memory management and JIT compilation. These aspects won't be explored further, which limits a rigorous conclusion. Nonetheless this points towards the JVM being responsible for the difference in performance.

4.5 Overall Evaluation

The results of the evaluation show that the Rejection-Inversion and Sysbench implementations are the most accurate and performant, closely followed by YCSB. The condensed lookup table is the fastest, but also less accurate. While currently the problems explored in the *frequency ratio graphs* exclude it from further consideration, it could be improved by increasing the precision of the table lookup so as to make it competitive with the other implementations. The FIO implementation is the least accurate and the slowest, which makes it unsuitable for further use. Lastly, the rejection algorithm is among the slowest and also not particularly accurate, which excludes it from any further consideration. All in all, YCSB and Sysbench are the most viable choices for further use, as they are both accurate and performant and cover the use cases of database and file system benchmarks respectively.

”

Abbreviations

PVLDB Proceedings of the VLDB Endowment

SIGMOD ACM Special Interest Group on Management of Data

CIDR Conference on Innovative Data Systems Research

OSDI USENIX Symposium on Operating Systems Design and Implementation

NSDI USENIX Symposium on Networked Systems Design and Implementation

FAST USENIX Conference on File and Storage Technologies

YCSB Yahoo! Cloud Serving Benchmark

FIO Flexible I/O Tester

DBMS Database Management System

TPC-C Transaction Processing Performance Council Benchmark C

RDBMS Relational Database Management System

PRNG Pseudo-Random Number Generator

TVD Total Variation Distance

KS Kolmogorov-Smirnov

List of Figures

1.1	Example of a Zipfian distribution	1
2.1	Probability Mass Function of a Zipfian distribution with $N = 100$ and $\alpha = 1$	4
2.2	Application of log on data from 2.1	5
2.3	To generate these graphs, a random uniform sample is taken for the x-axis as well as another normalized one for the y-axis.	7
2.4	Rejection-Inversion Sampling for a Zipfian distribution	8
2.5	The table contains the frequency of each index, whereas the array is populated with as many entries as there occurrences of each index. The sample "2" was generated, which is used to access the 2nd position in the array	9
2.6	Jim Gray's method for generating samples from a Zipfian distribution .	10
2.7	Accuracy Measure	11
2.8	Kolmogorov-Smirnov Test	12
3.1	Pie charts for each conference (excluding Web Polygraph and pgBench).	16
3.2	Comparison of Jim Gray's version (right) and the one employed by YCSB (left).	17
3.3	YCSB does sampling by inversion (1) and then hashes the value to spread in in the available range (2)	17
3.4	UML diagram of the benchmarking module	19
4.1	All distributions have support size 10^8 and have 10^9 samples	24
4.2	Log-log plots of the performance of the 6 different variate generators. .	26
4.3	All distributions have support size 10^6 and have 10^9 samples	27
4.4	Kolmogorov-Smirnov test results for the different samplers.	28
4.5	TVD test results for the different samplers.	29

List of Tables

2.1	Summary of methods for sampling from the Zipf-Mandelbrot distribution	6
3.1	Keyword search matrix	15
4.1	Computer Specifications	22
4.2	Software Specification	22
4.3	Interpreter Versions	22
4.4	Simulation specification for each accuracy criteria	23
4.5	Accuracy Results for 10^9 samples	30
4.6	Microbenchmarking Results	32

Bibliography

- [1] B. B. Mandelbrot, “Is Nature Fractal?” *Science*, vol. 279, no. 5352, pp. 783–783, Feb. 6, 1998. DOI: 10.1126/science.279.5352.783c.
- [2] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger, “Quickly generating billion-record synthetic databases,” *SIGMOD Rec.*, vol. 23, no. 2, pp. 243–252, May 24, 1994, ISSN: 0163-5808. DOI: 10.1145/191843.191886.
- [3] M. E. Crovella, “Performance Evaluation with Heavy Tailed Distributions,” in *Computer Performance Evaluation. Modelling Techniques and Tools*, B. R. Haverkort, H. C. Bohnenkamp, and C. U. Smith, Eds., Berlin, Heidelberg: Springer, 2000, pp. 1–9, ISBN: 978-3-540-46429-7. DOI: 10.1007/3-540-46429-8_1.
- [4] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013, ISSN: 0001-0782, 1557-7317. DOI: 10.1145/2408776.2408794.
- [5] D. M. W. Powers, “Applications and explanations of Zipf’s law,” in *Proceedings of the Joint Conferences on New Methods in Language Processing and Computational Natural Language Learning - NeMLaP3/CoNLL ’98*, Sydney, Australia: Association for Computational Linguistics, 1998, p. 151, ISBN: 978-0-7258-0634-7. DOI: 10.3115/1603899.1603924.
- [6] *Zipf’s law*, in *Wikipedia*, Feb. 12, 2025.
- [7] M. E. Kuhl, “History of random variate generation,” in *Proceedings of the 2017 Winter Simulation Conference*, ser. WSC ’17, Las Vegas, Nevada: IEEE Press, Dec. 3, 2017, pp. 1–12, ISBN: 978-1-5386-3427-1.
- [8] F. Radicchi, “Underestimating extreme events in power-law behavior due to machine-dependent cutoffs,” *Physical Review E*, vol. 90, no. 5, p. 050801, Nov. 3, 2014, ISSN: 1539-3755, 1550-2376. DOI: 10.1103/PhysRevE.90.050801. arXiv: 1405.0058 [physics].
- [9] L. Devroye, “Discrete Univariate Distributions,” in *Non-Uniform Random Variate Generation*, L. Devroye, Ed., New York, NY: Springer, 1986, pp. 485–553, ISBN: 978-1-4613-8643-8. DOI: 10.1007/978-1-4613-8643-8_10.

- [10] R. A. Kronmal and A. V. Peterson, "On the Alias Method for Generating Random Variables from a Discrete Distribution," *The American Statistician*, vol. 33, no. 4, pp. 214–218, Nov. 1979, issn: 0003-1305, 1537-2731. doi: 10.1080/00031305.1979.10482697.
- [11] W. Hörmann and G. Derflinger, "Rejection-inversion to generate variates from monotone discrete distributions," *ACM Transactions on Modeling and Computer Simulation*, vol. 6, no. 3, pp. 169–184, Jul. 1996, issn: 1049-3301, 1558-1195. doi: 10.1145/235025.235029.
- [12] G. Marsaglia, W. W. Tsang, and J. Wang, "Fast Generation of Discrete Random Variables," *Journal of Statistical Software*, vol. 11, no. 3, 2004, issn: 1548-7660. doi: 10.18637/jss.v011.i03.
- [13] *Rejection sampling*, in *Wikipedia*, Dec. 28, 2024.
- [14] D. E. Knuth, *The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms*. USA: Addison-Wesley Longman Publishing Co., Inc., Oct. 1997, 784 pp., isbn: 978-0-201-89684-8.
- [15] A. Clauset, C. R. Shalizi, and M. E. J. Newman, "Power-law distributions in empirical data," *SIAM Review*, vol. 51, no. 4, pp. 661–703, Nov. 4, 2009, issn: 0036-1445, 1095-7200. doi: 10.1137/070710111. arXiv: 0706.1062 [physics].
- [16] J. Leydold, "Error Detection in Non-Uniform Random Variate Generators,"
- [17] J. F. Monahan, "Accuracy in random number generation," *Mathematics of Computation*, vol. 45, no. 172, pp. 559–568, 1985, issn: 0025-5718, 1088-6842. doi: 10.1090/S0025-5718-1985-0804945-X.
- [18] G. Marsaglia, W. W. Tsang, and J. Wang, "Evaluating Kolmogorov's Distribution," *Journal of Statistical Software*, vol. 8, pp. 1–4, Nov. 10, 2003, issn: 1548-7660. doi: 10.18637/jss.v008.i18.
- [19] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [20] B. Keng. "The Empirical Distribution Function," Bounded Rationality. (Mar. 12, 2016), [Online]. Available: <http://bjlkeng.github.io/posts/the-empirical-distribution-function/> (visited on 03/14/2025).

- [21] L. Devroye, L. Györfi, and G. Lugosi, “Inequalities and Alternate Distance Measures,” in *A Probabilistic Theory of Pattern Recognition*, L. Devroye, L. Györfi, and G. Lugosi, Eds., New York, NY: Springer, 1996, pp. 21–37, ISBN: 978-1-4612-0711-5. DOI: 10.1007/978-1-4612-0711-5_3.
- [22] W. M. Bramer, G. B. de Jonge, M. L. Rethlefsen, F. Mast, and J. Kleijnen, “A systematic approach to searching: An efficient and complete method to develop literature searches,” *Journal of the Medical Library Association : JMLA*, vol. 106, no. 4, pp. 531–541, Oct. 2018, ISSN: 1536-5050. DOI: 10.5195/jmla.2018.283. PMID: 30271302.
- [23] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in *Proceedings of the 1st ACM Symposium on Cloud Computing*, ser. SoCC ’10, New York, NY, USA: Association for Computing Machinery, Jun. 10, 2010, pp. 143–154, ISBN: 978-1-4503-0036-0. DOI: 10.1145/1807128.1807152.
- [24] “1. fio - Flexible I/O tester rev. 3.38 — fio 3.38-19-gd1eb documentation.” (), [Online]. Available: https://fio.readthedocs.io/en/latest/fio_doc.html (visited on 01/16/2025).
- [25] A. Kopytov, *Akopytov/sysbench*, Jan. 16, 2025.
- [26] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux, “OLTP-Bench: An extensible testbed for benchmarking relational databases,” *Proc. VLDB Endow.*, vol. 7, no. 4, pp. 277–288, Dec. 1, 2013, ISSN: 2150-8097. DOI: 10.14778/2732240.2732246.
- [27] “Math – Commons Math: The Apache Commons Mathematics Library.” (), [Online]. Available: <https://commons.apache.org/proper/commons-math/> (visited on 02/22/2025).
- [28] “AuctionMark: An OLTP Benchmark for Shared-Nothing Database Management Systems « H-Store.” (), [Online]. Available: <https://hstore.cs.brown.edu/projects/auctionmark/> (visited on 02/22/2025).
- [29] “Perf: Linux profiling with performance counters.” (), [Online]. Available: <https://perfwiki.github.io/main/> (visited on 03/09/2025).
- [30] “Avoiding Benchmarking Pitfalls on the JVM.” (), [Online]. Available: <https://www.oracle.com/technical-resources/articles/java/architect-benchmarking.html> (visited on 03/09/2025).
- [31] “Google/benchmark: A microbenchmark support library.” (), [Online]. Available: <https://github.com/google/benchmark> (visited on 03/09/2025).

- [32] L. Deng and R. Chhikara, “Robustness of some non-uniform random variate generators,” *Statistica Neerlandica*, vol. 46, no. 2–3, pp. 195–207, 1992, ISSN: 1467-9574. DOI: 10.1111/j.1467-9574.1992.tb01337.x.
- [33] D. Beyer, S. Löwe, and P. Wendler, “Benchmarking and Resource Measurement,” in *Model Checking Software*, B. Fischer and J. Geldenhuys, Eds., Cham: Springer International Publishing, 2015, pp. 160–178, ISBN: 978-3-319-23404-5. DOI: 10.1007/978-3-319-23404-5_12.
- [34] M. E. Crovella and L. Lipsky, “Simulations with Heavy-Tailed Workloads,” in *Self-Similar Network Traffic and Performance Evaluation*, John Wiley & Sons, Ltd, 2000, pp. 89–100, ISBN: 978-0-471-20644-6. DOI: 10.1002/047120644X.ch3.
- [35] B. Jenkins. “Lookup3.” (), [Online]. Available: <http://burtleburtle.net/bob/c/lookup3.c> (visited on 03/19/2025).
- [36] “A Hash Function for Hash Table Lookup.” (), [Online]. Available: <http://burtleburtle.net/bob/hash/doobs.html> (visited on 03/19/2025).
- [37] “Benchmarking tips — LLVM 21.0.0git documentation.” (), [Online]. Available: <https://llvm.org/docs/Benchmarking.html> (visited on 03/11/2025).
- [38] J. Jenkov. “JMH - Java Microbenchmark Harness.” (), [Online]. Available: <http://jenkov.com/tutorials/java-performance/jmh.html> (visited on 03/12/2025).